7주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

교사용 진행안

7주차 코드 내려받기

198 · 단계별 힌트

LeetCode 198: House Robber · Medium

H1. 현재 위치를 선택할 때 앞의 어느 부분까지 최적해를 사용할 수 있나요?

H2. 이전 상태를 덮기 전에 어떤 값이 필요한가요?

H3. D[i]는 앞 i개 최댓값입니다. D[0]=0, D[1]=nums[0]이며 i>=2에서 D[i]=max(D[i-1],D[i-2]+nums[i-1]) 입니다. 코드에서는 이전 두 상태를 0으로 시작해 첫 값부터 같은 갱신을 적용합니다.

198 · 정당성과 비용

최적 선택은 현재 위치를 선택하거나 선택하지 않는 두 경우로 나뉩니다. 두 경우 모두 남은 접두부를 최적으로 풀어야 하므로 점화식이 모든 최적해를 포함합니다.

n=길이. O(n) 시간, O(1) 보조 공간. 선택 위치 복원은 별도 상태가 필요합니다.

198 · 반례와 검증

[2,1,1,2]에서 짝수 위치만 또는 홀수 위치만 합하면 최적 4를 놓칩니다.

`nums=[3,1,4,2] → 7`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

198 · 구현

```
def rob(nums):
    previous_two = previous_one = 0
    for value in nums:
        previous_two, previous_one = previous_one, max(previous_one, previous_two + value)
    return previous_one
```

322 · 단계별 힌트

LeetCode 322: Coin Change · Medium

H1. dp[a]는 방법의 수인가요, 동전의 수인가요?

H2. 불가능한 금액을 0으로 두면 어떤 후보가 생기나요?

H3. dp[0]=0, 나머지는 inf입니다. 작은 금액부터 dp[a-c]+1 중 최소를 취합니다.

322 · 정당성과 비용

최적해의 마지막 동전 c를 제거하면 a-c의 최적해가 남아야 합니다. 모든 가능한 마지막 동전을 비교하므로 최소를 놓치지 않습니다. 양수 동전이라 필요한 상태가 먼저 계산됩니다.

A=amount,m=종류 수. O(Am) 시간·O(A) 보조 공간. A의 비트 길이에 대해서는 의사다항 시간입니다.

322 · 반례와 검증

큰 동전 우선은 [1,3,4], 6에서 4+1+1을 골라 실패합니다. [2],3은 -1이고 amount=0은 0입니다.

`coins=[1,3,4], amount=6 → 2`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

322 · 구현

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for total in range(1, amount + 1):
        for coin in coins:
            if coin <= total:
                dp[total] = min(dp[total], dp[total - coin] + 1)
    return -1 if dp[amount] == float("inf") else dp[amount]
```

62 · 단계별 힌트

LeetCode 62: Unique Paths · Medium

H1. 도착 직전 위치는 어디일 수 있나요?

H2. 같은 행 객체를 여러 번 참조하면 갱신이 독립적인가요?

H3. 첫 행·열은 1이고 내부는 위쪽 수+왼쪽 수입니다. 행마다 새 list를 만듭니다.

62 · 정당성과 비용

각 경로의 마지막 이동은 아래 또는 오른쪽 하나입니다. 두 집합은 겹치지 않고 전체 경로를 덮으므로 수를 더합니다. 행 순서로 필요한 선행 상태가 준비됩니다.

R,C=행·열 수. O(RC) 시간·O(RC) 보조 공간. 큰 정수의 비트 비용은 별도이며 공식 답 범위에서는 작습니다.

62 · 반례와 검증

`[[1]*C]*R`은 같은 행을 공유합니다. `[[1]*C for _ in range(R)]`로 만듭니다.

`rows=2, cols=4 → 4`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

62 · 구현

```
def unique_paths(rows, cols):
    dp = [[1] * cols for _ in range(rows)]
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = dp[r-1][c] + dp[r][c-1]
    return dp[-1][-1]
```

1143 · 단계별 힌트

LeetCode 1143: Longest Common Subsequence · Medium

H1. 접두 문자열 길이 i,j는 어떤 부분 문제인가요?

H2. 끝 문자가 다르면 어느 끝 문자를 빼야 하나요?

H3. 같으면 대각선+1, 다르면 위·왼쪽 중 최대입니다. 길이 0 접두부의 행·열은 0입니다.

1143 · 정당성과 비용

끝 문자가 같으면 두 끝을 맞춘 최적 부분수열을 택할 수 있습니다. 다르면 둘을 동시에 마지막으로 맞출 수 없어 적어도 한쪽을 생략해야 합니다. 두 경우의 최댓값을 취합니다.

n,m=문자열 길이. O(nm) 시간·O(nm) 보조 공간. 길이만 반환하며 문자열 복원은 별도입니다.

1143 · 반례와 검증

부분수열은 문자의 순서를 유지하면서 중간 문자를 건너뛸 수 있습니다. “abc”에서 b를 건너뛰면 “ac”가 되므로 공통 부분수열의 길이는 2입니다.

```
first="abca", second="aca" → 3
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

1143 · 구현

```
def lcs(first, second):
    rows, cols = len(first), len(second)
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(1, rows + 1):
        for c in range(1, cols + 1):
            if first[r-1] == second[c-1]:
                dp[r][c] = dp[r-1][c-1] + 1
            else:
                dp[r][c] = max(dp[r-1][c], dp[r][c-1])
    return dp[rows][cols]
```

실행과 참고자료

```
cd code
python demo_13.py
python demo_14.py
python verify.py
```

표준 라이브러리 import는 `solutions.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

중간 점검의 답과 근거

1. 4. 첫 값과 마지막 값을 선택합니다. 홀수·짝수 위치의 합만 비교하면 놓칩니다.

2. 2. 3+3이며 큰 동전부터 고른 4+1+1보다 적습니다.

3. 1과 4. 두 번째 행은 왼쪽부터 1,2,3,4입니다.

4. 2. 부분수열은 순서를 보존하며 중간 문자를 건너뛸 수 있습니다.

개념 질문 해설

`dp[a-c]+1`의 1은 마지막으로 추가한 동전입니다. 금액이 0이면 동전 0개이고, 만들 수 없는 금액은 무한대로 유지합니다.

경로 수는 마지막 이동이 위·왼쪽인 두 집합의 크기를 더합니다. LCS는 가능한 길이 중 최댓값을 구하므로 같은 합 연산을 사용할 수 없습니다.